



DEPARTMENT OF OCEAN OPERATIONS AND CIVIL ENGINEERING

AIS4004 DIGITAL TWIN

Neural Network for SHM of the frame

Author:
William Haugland

April 2026

Table of Contents

Preface	i
List of Figures	ii
List of Tables	ii
1 Objective	2
2 Requirements	2
3 Method (AI or FEDEM?)	2
4 Model	2
5 Results and Discussion	3
5.1 parameters	4
5.2 Provided code and modifications	4
5.3 Visual Tuning	5
5.4 Simulation	7
5.5 Tuned Model	9
5.6 Training Convergence and Generalization	9
5.7 Comparing the tuned models	12
6 Conclusion	13
7 Recommendations	13
Appendices	i

List of Figures

5.1	Example of generalization and overfitting	5
5.2	Batch training multiple window lengths	6
5.3	Batch results showing Strides of 1, 2, 5 and 10	7
5.4	Batch results showing Batch sizes of 16, 24, 32, 48 and 64	7
5.5	Batch results showing Epochs of 10, 25, 40, 50, 75, 100	8
5.6	Batch results showing Learning rates of 0.005, 0.001, 0.0005, and 0.0001	8
5.7	Batch results showing Window length of 50, 75, 100, 125, 150	8
5.8	Training and Testing losses for Stride and Batch size	10
5.9	Training and Testing losses for Epochs and Learning rate	11
5.10	Training and Testing loss for Window Length	11
5.11	Tuning comparison showing the three different models from Figure 5.3	12
5.12	Training and Testing loss comparison of the tuned models	13

List of Tables

5.1	Experimental results for Neural Network parameter tuning across five runs.	5
5.2	Final parameter tuning	9
5.3	Final parameter tuning, FV is First Visual, TV is Tuned Visual and TL is Tuned Loss	12

DATE	28th April 2026	Classified ☐	Open ☒
TITLE	AIS 4004 Report "Neural Network for SHM of the frame"		
EMPLOYER	NTNU/IHB		
EMPLOYER REF.	Terje Rølvåg, AIS 4004		
COMPLETED BY	"William Haugland"		
SUMMARY Training a Neural network to predict gauge measurements gave promising results when tuned correctly. Finding the right parameters to predict gauge measurements was done by trial and error where the best performing parameter was selected before testing for another one. Narrowing down and comparing the results was tedious with the provided scripts, but modifying the code and adding a graph saver made the comparison a lot easier when comparing different parameters. The tuned model gave good prediction points up in till 120s, where all test gave inverted or noisy predictions. <i>The report should be between 5-10 pages (Note: less is sometimes more but I don't count pages ;-)</i> <i>regards Terje</i>			

1 Objective

The objective of this project is to develop a Reduced Order Model or ROM capable of predicting structural strains at critical locations on a bicycle frame using only non-intrusive motion data. The data variables are the acceleration in x,y and z as well as the angular acceleration in x,y,z directions. A Neural Network will be trained to make predictions for the output gauge stresses which are given from acceleration inputs.

By implementing this SHM system, Hardrox can monitor the health of the bicycle frame during real world usage without requiring strain gauges on their bicycles. This allows them to detect damage by identifying high stress events that could compromise structural integrity. The data from SHM can be used to optimize and build lighter yet stronger frames.

2 Requirements

The data, which will be analysed, have information from 6 MRU's, which gives 3-axis of acceleration and 3-axis of angular acceleration, and 6 strain gauges on different places on the frame. The data is time synchronized which means that both the inputs and outputs data give highly accurate input output correlation. The data represents a typical bicycle handling and riding manoeuvring, with both high-frequency and low-frequency structural responses. The data provided was one csv file which is used for both training and running the prediction. This approach often leads to domain generalization, where the NN can predict this specific ride it was trained on perfectly, but may struggle with other conditions not present in the dataset. It would therefore be advisable to train on more diverse data and have a different testing dataset when tuning the NN.

3 Method (AI or FEDEM?)

For this project it was decided to use a Neural Network based ROM. The project uses a 1D Convolutional Neural Network or CNN which uses a sliding window approach to capture the temporal context and non-linear dynamics of the frame's response to motion. FEDEM on the other hand uses a lookup table which is typically linear.

4 Model

The project is implemented in Python using PyTorch for the deep learning architecture. To make it more user friendly it uses Streamlit for an interactive GUI. For the architecture it uses a 1D CNN with three convolutional layers, where the layers are 34, 64 and 128 filters with ReLU activations.

The training of the model supports Nvidia cuda cores, which makes the process of training the model much faster if available. Doing training on a CPU is computationally expensive and time consuming and a GPU is able to do these kinds of processes much faster. The training time can be reduced by 20-50 times by enabling the use of CUDA cores and is the reason why AI data centers and bitcoin miners prefer using GPU's for these tasks.

Structural strain during a dynamic event (like a bicycle hitting a pothole or landing a jump) is not instantaneous, the frame’s response at time t is heavily influenced by the motion leading up to that moment due to inertia, damping, and vibration propagation.

To capture this physics, the method employs a Sliding Window technique. Instead of feeding the AI a single snapshot of the 6 MRU inputs, the model is fed a continuous block of time, for example a "Window Length" of 100 samples. If the sensors run at 100 Hz, this represents 1 full second of dynamic history. The network uses this 1-second history to predict the exact strain gauge values at the end of that window. The window is shifted forward by a set Stride, for example 10 samples, to generate the next training example, ensuring the network learns from heavily overlapping, continuous dynamic events.

The core of the method is a PyTorch-based 1D CNN, which is highly optimized for time-series signal processing. The architecture works in two main stages. Feature Extraction or Convolutional Layers, the network applies three layers of 1D convolutions, with filter sizes of 32, 64, and 128, across the time window. These act as learnable digital filters. The early layers learn to detect simple signal patterns, like sudden acceleration spikes or sharp changes in angular rate, while deeper layers combine these into complex multi-axis dynamic features like breaking. Regression Head or Fully Connected Layers, after the temporal features are extracted and pooled, a fully connected neural network translates these abstract mathematical features into 6 distinct, physical strain values corresponding to the gauge locations on the bicycle frame.

Bicycle telemetry data is inherently noisy and filled with extreme outliers. The method addresses this using two specific techniques, Robust Scaling and Huber Loss Function. Standard normalization can be ruined by massive spikes in the data. The method uses a RobustScaler to normalize the inputs and outputs, ensuring that extreme shocks don’t skew the baseline training. During training, the network is optimized using HuberLoss instead of standard Mean Squared Error or MSE. Huber loss acts like MSE for small errors but shifts to a linear penalty for large errors. This prevents the AI from over-correcting its weights when it encounters a massive, unpredictable spike in the training data, leading to a much more stable model.

The primary alternative to this AI approach is generating a pre-calculated ROM look-up table using FEDEM. The limitation of a FEDEM ROM is the need for a look-up table, typically constrained to linear structural responses. The advantage of the Neural Network is that the dynamic loading of a bicycle frame is highly non-linear. Factors like tire compression, rider body-mass damping, frame flex, and complex multi-axis load combinations cannot be easily simplified into a linear table. The deep learning AI method, through its non-linear activation functions (ReLU), naturally maps these complex, non-linear relationships directly from empirical real-world test data, resulting in a much more accurate prediction of peak stresses during extreme ride events.

5 Results and Discussion

This section encompasses the result and discusses from the project, explaining what the parameters are and what scripts were modified to make graph saving and batch running possible.

5.1 parameters

The objective revolves around tuning parameters for the model to make it generalize the inputs to the outputs. The parameters of Window Length, Stride, Batch size, Epochs and Learning Rate are to be changed to tune the model to make the best prediction of outputs given a set of inputs. Window length is the number of historical data points the model looks at to make a single prediction. If the sampling rate is 100hz, a window of 100 samples means the model looks at the last 1 second of data. Stride defines how many samples the sliding window moves forward after each training example. A stride of 10 means that the model takes a window, moves forward 10 samples and take the next window. A smaller stride creates more overlapping training data but increases the training time.

Epochs are how many complete runs of the training data is passed through the NN. By having more Epochs the model is allowed to learn deeper, but it might be in risk of overfitting or memorizing the training data instead of learning it. Batch size is the number of training examples processed simultaneously in one iteration. Larger batches are faster to process at the cost of memory. The last parameter is the learning rate. this controls how much the models weights are adjusted in response to error or loss. When the learning rate is too high it might overshoot the optimal solution and if it's too low, the training will be slow or get stuck not finding the best solution.

As a training output the model uses HuberLoss, which is a hybrid between Mean Squared Error or MSE, and Mean Absolute Error or MAE. HuberLoss is less sensitive to outliers in the data than MSE and this helps if the sensor data is noisy or has glitches. A seed is used as a starting point for the weights. By keeping the seed the same, the training will consistently start with the same weights to have a consistent comparison.

5.2 Provided code and modifications

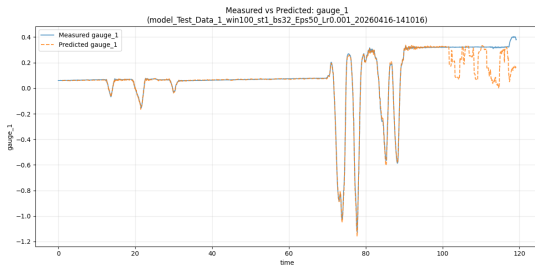
The provided code showed plots after running the trained data on the Streamlit website. These plots are useful to compare the impact of different parameters, but the plots were never saved to the project as a PNG and would be lost after running another trained dataset. This is a problem when trying to compare different training runs alone. The first step to saving the plots was to use matplotlib and a helper script to save the graphs from training and running of the model. This was useful to compare with different gauges and parameters. The training and running of the data was done with three students and on three machines to make the train/running faster for the comparison.

5.3 Visual Tuning

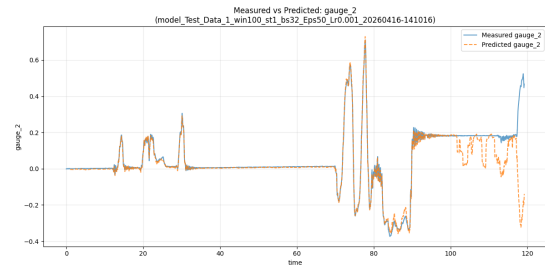
	Window Length	Stride	Batch size	Epocs	Learning Rate
Run 1 Stride	100	$\begin{matrix} 5 \\ \underline{2} \\ 1 \end{matrix}$	32	50	0.001
Run 2 Batch size	100	2	$\begin{matrix} \underline{16} \\ 24 \\ 32 \\ 40 \\ 48 \\ 64 \end{matrix}$	50	0.001
Run 3 Epocs	100	2	16	$\begin{matrix} 25 \\ 40 \\ 50 \\ 75 \\ \underline{100} \end{matrix}$	0.001
Run 4 Learning Rate	100	2	16	100	$\begin{matrix} \underline{0.0001} \\ 0.0005 \\ 0.001 \end{matrix}$
Run 5 Window Length	$\begin{matrix} 50 \\ 75 \\ 100 \\ \underline{110} \\ 125 \\ 150 \end{matrix}$	2	16	100	0.0001
Final tune	110	2	16	100	0.0001

Table 5.1: Experimental results for Neural Network parameter tuning across five runs.

Table 5.1 shows the results from the first set of training runs of the NN. The method was to keep all parameters the same and train the model on one different parameter. This was done to isolate the parameters during tuning, but doesn't necessarily lead to the best performing model. Run one gave the best results with a Stride of 2. Run two showed that the best results occurred with a Batch size of 16. Run three showed that larger Epochs of 100 gave the best results. Run four gave the best results with the standard 0.0001 Learning rate, and the last run five for Window Length gave the best results with 110.



(5.1a) Gauge 1



(5.1b) Gauge 2

Figure 5.1: Example of generalization and overfitting

Figure 5.1 shows the model for Run 1, where the model generalized and failed at the end. All models had a hard time with the last 15% of the dataset even if it generalized to the first 85%. The training and running is done on the same dataset, where the training is missing the last 15% and has never seen the last part. This could mean that the model is overfitting and just learning what the outputs are, not predicting it from the inputs. The tuning of the model could work very well on this dataset, but fall apart with a different dataset. The model has essentially memorized its exam or testing instead of learning how to process it. These results were compared from visual inspection of the graphs to find the best performing results.

After implementing the graph saving, another problem accrued. How to show the differences in the report without having 20 different graphs. The training script was modified with help from Gemini to add batch training and running. As this task is not a coding exercise and the objective was to tune the model with different parameters, Gemini was used to quickly modify the code to allow for graphing and batch training for easier comparison. The code was verified by me before implementation.

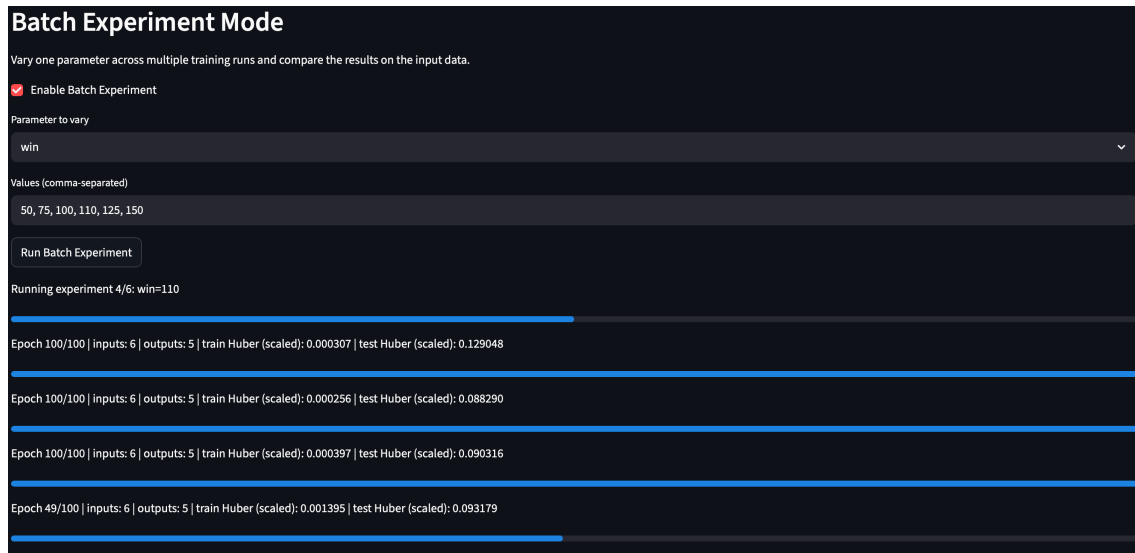
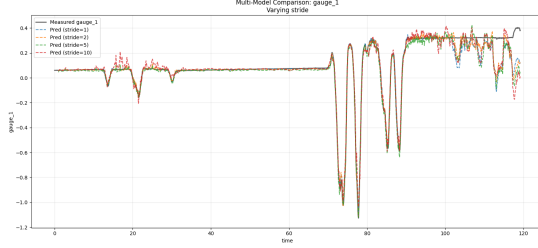


Figure 5.2: Batch training multiple window lengths

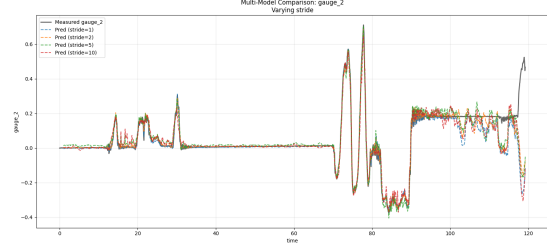
[] *Train_SMH_Neural_Network.py* was modified to be able to train and run batches of different parameters and save plots showing the entire batch training, as seen in Figure 5.2. The basic features from the original script is still available. Some helping scripts were also constructed to aid in the graph generation. This makes it possible to have window length, batch size etc on the same and train/run the model with different strides of 1, 2, 5 and 10 and plot it in to the same graph inside a auto generated labelled folder. When the model trains for a single parameter it automaticity starts processing the .pt (the model) before starting on the next training for the next selected parameter, keeping all other parameters the same. This makes it easy to store and compare the models performance with a single graph, something which was not possible with the provided script. The core Neural Network is the same but the modifications was done as a ease of use for the report.

5.4 Simulation

For the modified batch simulation the runs were done similarly as shown in Table 5.1, where the starting point was 100 Window Length, Batch size of 32, 50 Epochs and Learning Rate of 0.001. After the model was trained and ran with Streamlit, it produced the graphs shown in Figure 5.3. The report focus on showing Gauges 1 and 2, but for the choosing the best performing parameter all gauges were inspected and compared.



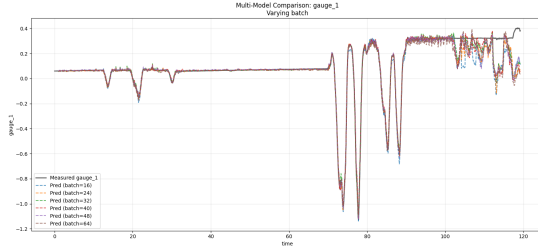
(5.3a) Gauge 1



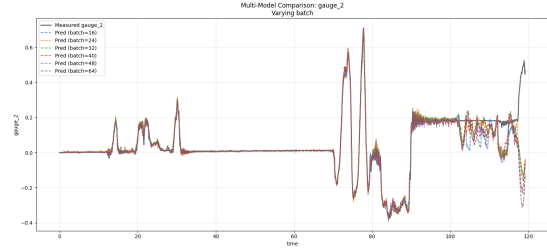
(5.3b) Gauge 2

Figure 5.3: Batch results showing Strides of 1, 2, 5 and 10

Figure 5.3 shows how the model predicts the output for Gauge 1 and 2. Strides of 5 and 10 has problems with following the straight lines and either over or undershoots the peaks. Stride of 1 is more precise but has a some tendencies to overshoot, even if it tracks the line well. Stride of 2 is not as good at following the line as stride of 1, but is almost perfect with the rapid frequencies. Stride of 2 was choose as the best candidate to keep for the next parameter. This is because the higher Stride is faster and might be better with longer Epochs.



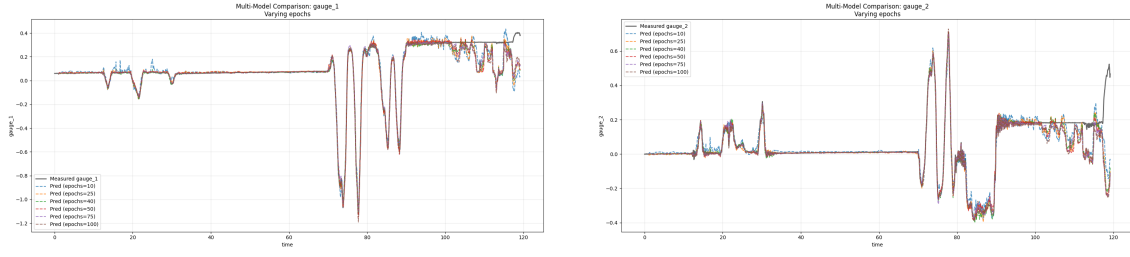
(5.4a) Gauge 1



(5.4b) Gauge 2

Figure 5.4: Batch results showing Batch sizes of 16, 24, 32, 48 and 64

Figure 5.4 shows the results with different Batch sizes of 16, 24, 32, 48 and 64, where all results where promising. The Batch size of 16 is the worst performing one with both under and overshooting. Batch sizes from 32 and up gave accurate predictions but the 24 batch size was better for rapid frequencies even if there where some overshoots. 48 and 64 was very accurate for the straight lines but struggles some more with these rapid frequencies. Batch size of 24 was chosen to train the model further.

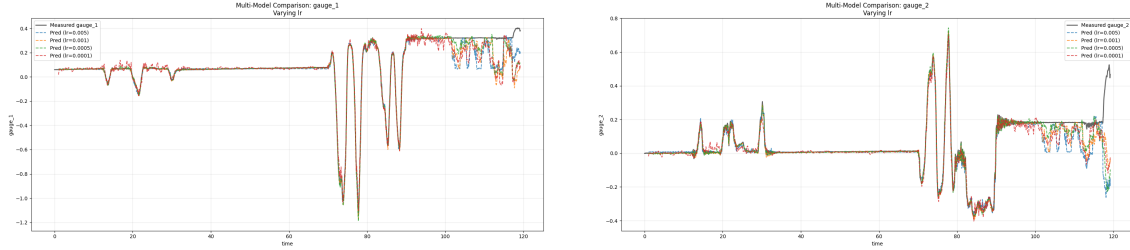


(5.5a) Gauge 1

(5.5b) Gauge 2

Figure 5.5: Batch results showing Epochs of 10, 25, 40, 50, 75, 100

Figure 5.5 shows the impact on longer and reruns of the training data on the model. Epochs below 75 is showing deviations on straight lines and peaks, were an epoch of 10 is absolutely the worst performing one with random spikes and overshoots. The longer training of the 75 and 100 shows a much closer prediction, were the 100 epochs is almost the same as the measured data. The 75 epochs had some overshoots at the peaks, especially for Gauge 1, but shows promise for approximation if training time is considered. Epochs of 100 were selected before training for the Learning rate.

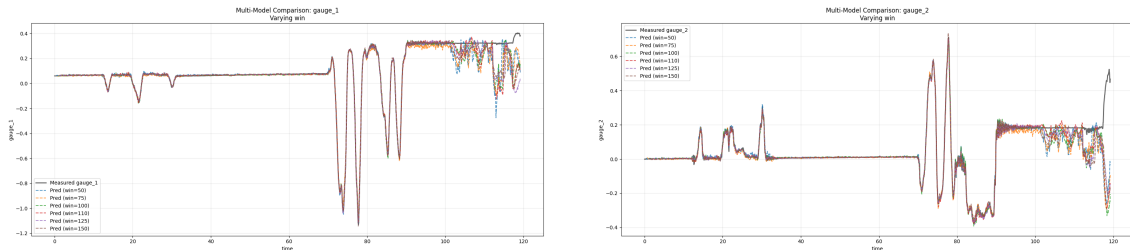


(5.6a) Gauge 1

(5.6b) Gauge 2

Figure 5.6: Batch results showing Learning rates of 0.005, 0.001, 0.0005, and 0.0001

Run 4 in Figure 5.6 shows how the Learning rate impacts the models predictions. The lower learning rate of 0.005 gave unstable straight lines, over and undershoots. This means that the learning rate was too low to predict with accuracy and it updated its weights too little. Learning rate of 0.001 gave good predictions with some overshooting in high frequencies, while 0.0005 is better at some points in gauge 2, but tends to over and undershoot places where 0.001 doesn't. The Learning rate of 0.0001 is giving jumpy predictions, which could indicate that it's updating its weights too often and fails to generalize. Learning rate of 0.0005 was chosen as the best tuned.



(5.7a) Gauge 1

(5.7b) Gauge 2

Figure 5.7: Batch results showing Window length of 50, 75, 100, 125, 150

The last run shown in Figure 5.7 gives an example of a well tuned system where the differences in window length are small. Mainly the worst performers are Window Lengths below 125, where they fail at high frequencies. The best performer is the one with Window Length of 125, where it keeps similar to the measured stresses, and where 150 sometimes overshoots at peaks.

5.5 Tuned Model

The results of the project are shown below in Table 5.3. As these were visual inspections of the performance of each parameter, the model tuning is not perfect, but it shows good generalization for most of the gauges.

	Window Length	Stride	Batch Size	Epochs	Learning Rate
First parameters	110	2	16	100	0.0001
Tuned parameters	125	2	24	100	0.0005

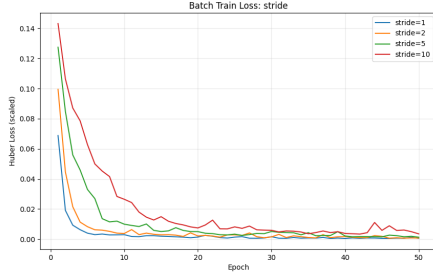
Table 5.2: Final parameter tuning

Table 5.3 shows the tuned parameters of the model. This is a bit different than the model from Table 5.1 where the Batch size of 16, Window Length of 110 and Learning rate of 0.0001 seemed to give the best model. The differences of the first and second tuned model can be explained that the results were visually inspected from multiple graphs, where small deviations are harder to see. The combined plots are also messy, making it hard to see where the actual measurements are, compared to the predicted ones. Either way, both models performed well after tuning and approximated the outputs from the inputs.

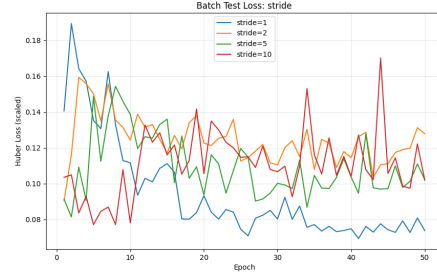
5.6 Training Convergence and Generalization

To ensure the Neural Network was learning the underlying physics of the bicycle frame rather than simply memorizing the dataset, the training and testing Huber loss curves were analysed for each parameter batch. The training loss indicates how well the model learns the data it is actively studying, while the test loss acts as a critical measure of the model's performance on unseen data. From the training and testing losses, the last parameter tuning will be tested.

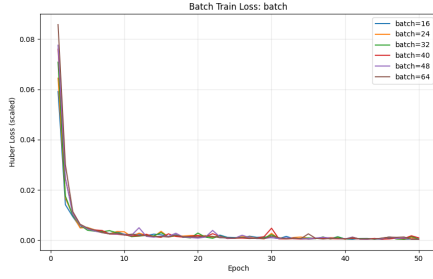
By generating consolidated batch loss plots, it became possible to visually track the convergence of the model and identify the exact epoch where the network stopped generalizing and began to overfit. For instance, while a higher number of Epochs continuously reduced the training error towards zero, the batch test loss plots often revealed a plateau occurring much earlier. This plateau indicates that further training provides diminishing returns for real-world generalization, as the model starts to memorize the specific noise and bumps of the training data instead of learning the general multi-axis dynamics. Instead of relying solely on visual approximations from messy, overlapping graphs, the raw training data could be extracted to precisely pinpoint the hyperparameters that resulted in the lowest, most stable test loss, confirming the visual findings presented in the earlier sections.



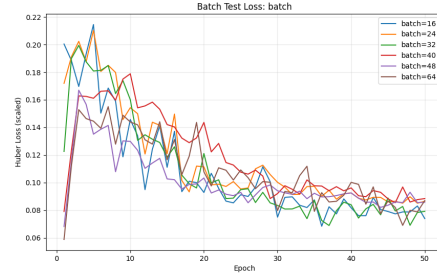
(5.8a) Stride Training Loss



(5.8b) Stride Testing loss



(5.8c) Batch size Training loss

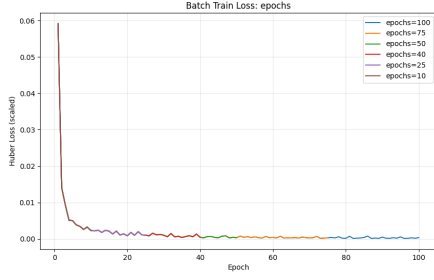


(5.8d) Batch size Test loss

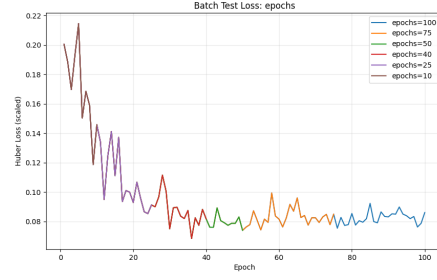
Figure 5.8: Training and Testing losses for Stride and Batch size

Figure 5.8 shows the training and testing losses of different Strides and Batch sizes. For smaller strides the model converges to a smoother lower error. This is expected as a smaller stride generated larger, more overlapping dataset, providing the model with more continuous temporal context. Overfitting seems to not be a problem as the Huber loss is rather small for all strides, but a stride of one converges to the lowest error. Since the Stride of 1 performed absolutely the best for both training and testing losses.

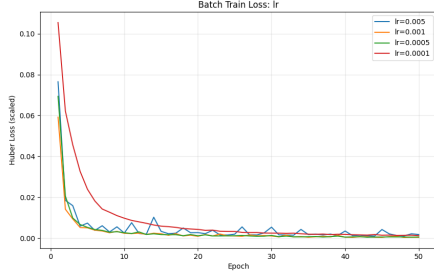
With a stride of 1, the Batch size training loss where all smoothly converging to a low error, with lower Batch sizes generalizing faster, and the most stable were 16 and 32. For the Huber loss the worst performers are 40 and 64, and 16 and 32 generalizes early and are stable at longer Epochs. Batch size of 24 and 48 reduces the error fast, but has some larger spikes and is beaten by 16 and 32 as the Epochs expand. The best performer for both early generalization and stable performance was batch size of 16.



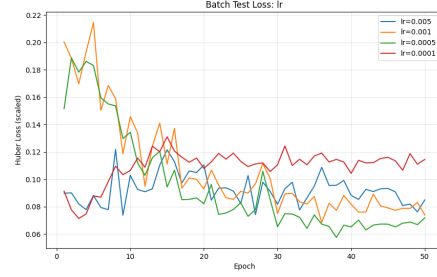
(5.9a) Epochs Training Loss



(5.9b) Epochs Testing loss



(5.9c) Learning Rate Training loss

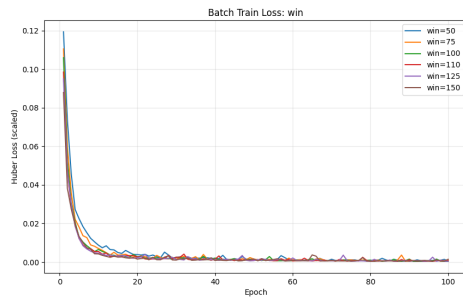


(5.9d) Learning Rate Test loss

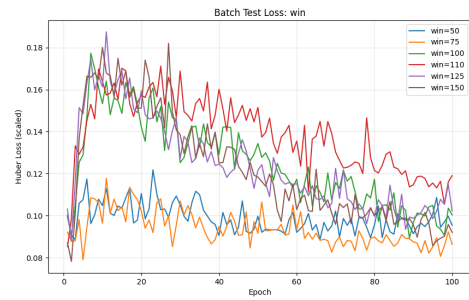
Figure 5.9: Training and Testing losses for Epochs and Learning rate

Figure 5.9 show the training and test losses for Epochs and Learning rates. The performance of the epochs are excellently the same, but longer epochs has more runs to train. Showing fast generalization and the lowest point of 36 Epochs. It was decided that 50 Epochs would be a great middle point as the 30-50 range is the most stable, and takes half as long as 100 Epochs to train.

The learning rate is not as one dimensional as the Epochs, where the lower learning rate of 0.0001 performed worse with highest training losses and increasing test losses before flattening out. This could be explained with too rapid weight updating. The best performer was 0.0005 with 0.001 as a close runner up, as both generalized fast and flattened out after 30 Epochs.



(5.10a) Window Length Training loss



(5.10b) Window Length Test loss

Figure 5.10: Training and Testing loss for Window Length

Figure 5.10 shows the performance of different Window Lengths. During training all models generalized well, but during testing the worst Window Length was 110. Both 50 and 75 has a low loss from the start, but 75 has a lower score at the end and as an average.

The final running for the loss analysis of the model is Window Length of 75, Stride of 1, Batch size of 16, Epochs of 50 and Learning rate of 0.0005.

The final three models are shown in Table 5.3.

5.7 Comparing the tuned models

To evaluate and find the best model it good to compare them side by side and show how they perform. All models are good performers but which tuning method gave the best predictions.

Model	WL	Stride	BS	Epochs	LR	Train Loss	Test Loss
FV Parameters	110	2	16	100	0.0001	0.00125	0.12974
TV Parameters	125	2	24	100	0.0005	0.00083	0.10533
TL Parameters	75	1	16	50	0.0005	0.00049	0.08845

Table 5.3: Final parameter tuning, FV is First Visual, TV is Tuned Visual and TL is Tuned Loss

Tabel 5.3 shows the three best performers after tuning. The next part is to run them together and check which has absolutely the best performance in both generalization and losses(error). The Tuned Visual reduces the train loss by 33.5% and test loss by 18.8% compared to the First Visual tuning, which is a significant reduction. The best performer Tuned from loss, reduced the training loss by 60.7% and training loss with 31.8% from the First visual tuning. This shows a significant reduction even compared to the second best model.

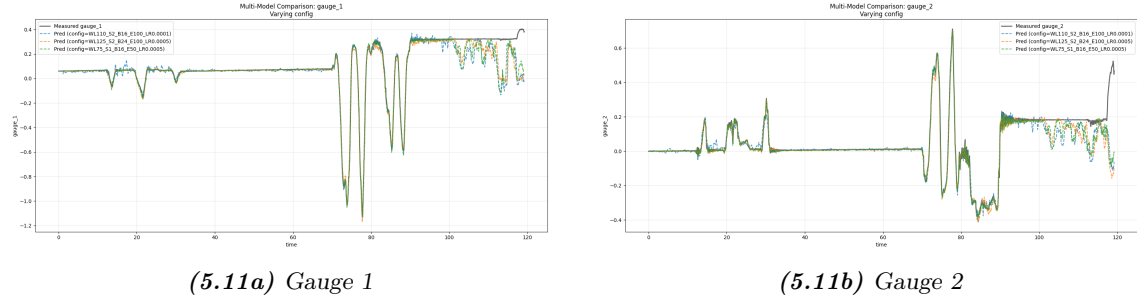
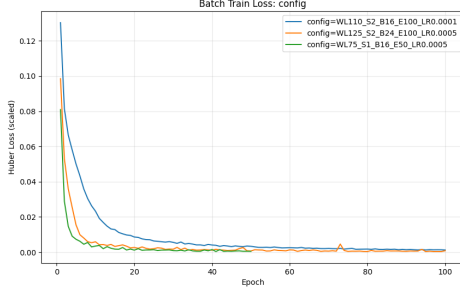
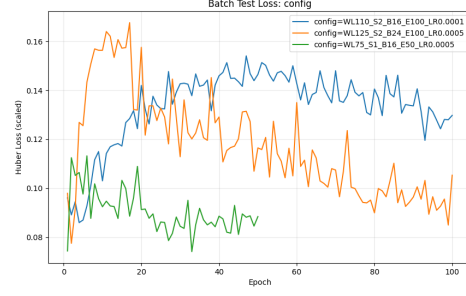


Figure 5.11: Tuning comparison showing the three different models from Figure 5.3

From Figure 5.11 the results from both gauges gave a clear picture of best performer. The First Visual tuning fails at straight lines after dips and is less stable in straight lines then the others. The Tuned Visual generalizes better and avoids most deviations at straight lines, but under and overshoots at the peaks. The best performer by far was from tuning with the loss data. It predicts most of the model correctly even at high frequencies and the peaks, but has some tendencies to overshoot at longer peaks.



(5.12a) Tuning comparison Training loss



(5.12b) Tuning comparison Test loss

Figure 5.12: Training and Testing loss comparison of the tuned models

Figure 5.12 shows similar results as Figure 5.11, where the Loss tuning provided the best results for both training loss and test loss. It generalized the fastest and has showed lower testing losses even at very few Epochs. The Loss tuning is definitely the best performer without compromising performance or training time.

6 Conclusion

The development of a Neural Network-based Reduced Order Model, or ROM, successfully demonstrated that a 1D Convolutional Neural Network can effectively predict non-linear structural stresses on a bicycle frame using only non-intrusive motion data, providing a robust alternative to linear FEDEM-based look-up tables. Through systematic hyperparameter tuning and the implementation of automated batch processing, it was determined that the model's performance is highly sensitive to input parameters. The optimal configuration was with a Window Length of 75, a Stride of 1, a Batch size of 16, 50 Epochs, and a Learning Rate of 0.0005 yielded the best balance of temporal context and stable convergence while preventing overfitting. Furthermore, utilizing Huber Loss and Robust Scaling proved essential for handling noisy telemetry data and extreme outliers. Ultimately, by shifting from manual visual inspection to a rigorous analysis of training and test loss curves, the project identified a robust model capable of accurately capturing complex, dynamic structural responses, confirming the viability of this AI-driven approach for real world structural health monitoring.

7 Recommendations

I really liked the project and i know the report became a lot longer then what i first envisioned. Trying to modify the training and running scripts to generate the plots and graphs which i wanted to show off were a fun project to work on and i learned a lot for parameter tuning of 1D CNN. The first scope of the project were just to visually check for best performances and trying to find fitting parameters for the next set of selections. Working together with others students for the first part to tune and get others incite on what they thought was the best performers. 1

Expanding on this thinking of how to visualize all the plot in to one and creating a batch running with comparison saving plots were fun to figure out and work with. Learning a

bit more about the test and training losses (error) gave me another view on model training to ensure the model is not just memorizing the training data. The project took more time than i originally wanted but it was fun and i think relevant for both my masters and other courses.

Appendices